



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

HOSPITAL APPOINTMENT SCHEDULING USING GREEDY ALGORITHM

A CAPSTONE PROJECT REPORT

**A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of
CSA0609 - DESIGN AND ANALYSIS OF ALGORITHMS**

**to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY**

Submitted by

Yuvasri B (192421231)

Under the Supervision of

DR. P. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Hospital Appointment Scheduling using Greedy Algorithm” has been carried out by Yuvasri B (192421231) under the supervision of Dr. P. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

The work presented in this report focuses specifically on Greedy Appointment Scheduling and Optimization, including the greedy selection strategy, implementation, testing, optimization analysis, and web-based implementation evidence.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez

Professor

Department of CSE

SIMATS Engineering

SIMATS, Chennai – 602105

COURSE FACULTY

Dr. P. Solainayagi

Professor

Department of CSE

SIMATS Engineering

SIMATS, Chennai - 602105

DECLARATION

I, Yuvasri B of the Department of Information Technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Hospital Appointment Scheduling using Greedy Algorithm” is the result of my own bonafide effort. The work presented in this report describes my individual contribution to Greedy Appointment Scheduling and Optimization. To the best of my knowledge, the implementation, algorithmic analysis, testing, observations, and documentation presented herein are original to my project work and have been prepared in accordance with academic integrity and engineering ethics.

I further declare that the patient records and appointment information shown in the application are demonstration data used for project implementation and testing. No real patient medical information is intended to be represented in this report.

Place: Chennai

Date: 7.09.2026

Signature of the Student with Name

Yuvasri B

INDIVIDUAL CONTRIBUTION STATEMENT

Student / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
Ramya S S (192421226)	Developed the greedy appointment scheduling logic; handled patient priority and preferred-time inputs and scheduling workflow.	Designed and implemented Module 2: Greedy Appointment Scheduling and Optimization and integrated the scheduling logic with the web interface.	Tested priority-based scheduling, preferred-slot allocation, slot availability, and scheduled/unscheduled patient outcomes.	Prepared Module 2 documentation, algorithm explanation, testing results, analysis, and supporting screenshots.	50%
Yuvasri B (192421231)	Developed patient registration and management functionality; handled patient data input and registration workflow.	Designed and implemented Module 1: Patient Registration and Management and contributed to the web interface design.	Tested patient registration, data validation, patient records, and module functionality.	Prepared Module 1 documentation, methodology, testing results, analysis, and supporting screenshots.	50%
Total					100%

ABSTRACT

Hospital appointment scheduling is a practical optimization problem in which a limited number of appointment slots must be assigned to patients with different levels of medical priority. Manual scheduling can lead to inefficient slot usage and may delay higher-priority patients. This project implements a greedy appointment scheduling approach that processes patients according to priority and assigns the best currently available slot.

The individual contribution presented in this report is Module 1 is responsible for registering patients and collecting relevant information such as patient identification, name, age, appointment requirement, urgency level, preferred doctor, and preferred time. If the preferred slot is already occupied, the earliest available slot is assigned. When no slot remains, the patient is marked as unscheduled. This produces a simple, deterministic, and easy-to-explain scheduling process.

The implementation was integrated into the SmartCare Hospital web application. The interface provides a patient registration workflow, a registered-patient view, a greedy scheduler, an appointment timeline, and an algorithm explanation page. The application screenshots demonstrate the implementation and the resulting schedule. The module also reports the number of scheduled and unscheduled patients and calculates slot utilization, supporting direct evaluation of scheduling efficiency.

Keywords: Greedy Algorithm, Appointment Scheduling, Priority Scheduling, Time Slots, Optimization, Healthcare Application, Python

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	Certificate	i
ii	Declaration	ii
iii	Individual Contribution Statement	iii
iv	Abstract	iv
v	Table of Contents	v
vi	List of Figures	vi
vii	List of Tables	vii
viii	List of Abbreviations / Symbols	viii
1	CHAPTER 1 – INTRODUCTION AND ENGINEERING PROBLEM	1
1.1	Background	1
1.2	Need for the Project	2
1.3	Problem Statement	2
1.4	Project Aim	3
1.5	Project Objectives	3
1.6	Scope	3
1.7	Expected Engineering Outcome	4
2	CHAPTER 2 – LITERATURE REVIEW AND EXISTING SOLUTIONS	5
2.1	Review Method	5
2.2	Review of Existing Work	5
2.3	Comparative Analysis	6
2.4	Research / Engineering Gap	7
2.5	Proposed Contribution	7
2.6	Module Architecture	8
3	CHAPTER 3 – ENGINEERING DESIGN, TRADE-OFFS, SAFETY, RISK & SECURITY	9
3.1	Requirements	9
3.2	Constraints & Applicable Standards	10
3.3	Design Specifications	10
3.4	Alternative Solutions & Evaluation	11
3.5	Selected Design & Detailed Design	12
3.6	Trade-off Analysis	13
3.7	Sustainability, Ethics, Safety, Risk & Security	14
4	CHAPTER 4 – IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	15
4.1	Development Approach & Resources	15
4.2	Software Implementation	16
4.3	Test Plan & Verification	19
4.4	Results	20
4.5	Data Analysis & Engineering Judgment	21
4.6	Comparison with Existing Solutions & Achievement of Objectives	22

4.7	Strengths & Limitations	23
4.8	Project Planning, Roles & Budget	24
5	CHAPTER 5 – CONCLUSION, FUTURE WORK AND REFLECTION	25
5.1	Conclusion	25
5.2	Future Work	26
5.3	Professional Learning & Individual Reflection	26
References	References	28
Appendix A	Detailed Algorithm / Pseudocode	29
Appendix B	Module Workflow	30
Appendix C	Source Code	31
Appendix D	Result Evidence	34

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System Architecture of Hospital Appointment Scheduling	11
2	SmartCare Hospital dashboard	17
3	Greedy scheduling result	17
4	Appointment schedule timeline	18
5	Greedy algorithm explanation	18
6	Registered Patient Details	19

LIST OF TABLES

Table No.	Table Name	Page No.
1	Individual Contribution Areas	iii
2	Functional and Non-Functional Requirements	9
3	Technology and Module Requirements	10
4	Design Alternatives and Evaluation	11
5	Greedy Scheduling Rules	12
6	Risk Register	14
7	Test Cases and Results	19
8	Result Summary	20
9	Project Milestones	24

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
DAA	Design and Analysis of Algorithms	N/A
UI	User Interface	N/A
API	Application Programming Interface	N/A
JSON	JavaScript Object Notation	N/A
HTTP	HyperText Transfer Protocol	N/A
n	Number of patients	Count
m	Number of available time slots	Count
$O(n \log n)$	Time complexity of sorting	Complexity
$O(nm)$	Worst-case scheduling time with linear slot search	Complexity

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Hospitals receive appointment requests from patients whose medical needs can differ greatly in urgency. When the number of available appointment slots is limited, the scheduling system must decide which patient should receive which slot. A useful scheduling policy should ensure that urgent cases are considered first while still making effective use of the available appointment times.

The Hospital Appointment Scheduling using Greedy Algorithm project addresses this problem using a priority-aware greedy strategy. Each patient has a priority level, a preferred appointment time, and a selected doctor. The scheduling module processes patients from the highest priority to the lowest priority and makes the best locally available decision for the patient currently being processed.

My individual contribution is Module 1 is responsible for registering patients and collecting relevant information such as patient identification, name, age, appointment requirement, urgency level, preferred doctor, and preferred time. The implementation maintains separate scheduled and unscheduled collections and reports slot utilization after processing all patients.

1.2 Need for the Project

Manual appointment allocation becomes difficult when many patients compete for a small number of slots. A first-come-first-served approach can place lower-priority patients ahead of urgent cases, while arbitrary assignment can leave preferred slots unused. A systematic scheduling rule is therefore required.

- Prioritize Emergency cases before High, Medium, and Low cases.
- Respect a patient's preferred time whenever it is still available.
- Use the earliest remaining slot when the preferred slot is unavailable.
- Prevent two patients from receiving the same appointment slot.
- Provide a measurable summary of scheduled, unscheduled, and utilized slots.

1.3 Problem Statement

Given a list of registered patients and a finite ordered list of appointment slots, assign each patient at most one slot so that higher-priority patients are considered first. For each patient, the preferred slot should be selected when available; otherwise, the earliest available slot should be selected. If no slot

remains, the patient must be recorded as unscheduled. The solution should be deterministic, easy to implement, and suitable for integration with a hospital web application.

1.4 Project Aim

To design and implement a greedy appointment scheduling module that allocates limited hospital appointment slots according to patient priority and preferred time while maximizing practical slot utilization.

1.5 Project Objectives

1. Develop a priority ordering from Emergency to Low.
2. Sort registered patients before making scheduling decisions.
3. Assign a patient's preferred slot whenever possible.
4. Assign the earliest available fallback slot when the preferred slot is occupied.
5. Maintain scheduled and unscheduled patient lists without duplicate slot allocation.
6. Calculate slot utilization and display the scheduling result through the web interface.
7. Analyze the time and space complexity and verify the implementation through test cases.

1.6 Scope

The module focuses on appointment-slot allocation rather than medical diagnosis or clinical decision making. It assumes that patient registration data and a set of valid appointment slots are already available. The greedy scheduler does not attempt to predict patient waiting times, optimize doctor workload globally, or solve a multi-day hospital timetable. Its scope is the deterministic allocation of a single ordered slot pool based on priority and preferred time.

- Input: registered patient records and available time slots.
- Processing: priority sorting and greedy slot selection.
- Output: scheduled patients, unscheduled patients, assigned slots, and utilization statistics.
- Interface: SmartCare Hospital web application for registration, scheduling, and result visualization.

1.7 Expected Engineering Outcome

The expected outcome is a working scheduling component that demonstrates how a classical greedy algorithm can be applied to a practical healthcare appointment problem. The module should give urgent patients earlier consideration, avoid slot duplication, provide a clear schedule, and generate a quantitative utilization measure. The implementation should also be understandable enough to support explanation during project demonstration and viva evaluation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review for this module focused on algorithmic scheduling concepts rather than clinical protocols. Priority scheduling, greedy choice strategies, appointment-slot allocation, and web-based scheduling interfaces were considered as the main areas. The purpose was to identify a simple approach that could be implemented reliably within the project time and integrated with the existing patient-registration module.

2.2 Review of Existing Work

Traditional appointment systems commonly use fixed time slots and may rely on first-come-first-served ordering. Such systems are straightforward but do not explicitly distinguish urgency. Priority-based scheduling improves this by assigning service preference to more urgent cases. Greedy algorithms are suitable when a locally best decision can be made repeatedly without revisiting earlier assignments.

For this project, the greedy rule is deliberately simple: sort patients by priority, then select the preferred slot if it is available; otherwise select the earliest remaining slot. The method is transparent and can be explained directly from the code, which is valuable for an academic implementation.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
First-Come-First-Served	Simple and predictable	Urgent patients may wait behind earlier registrations	Motivates explicit priority ordering
Manual Scheduling	Can use human judgment	Slow, inconsistent, and difficult to scale	Reduced through automated allocation
Round-Robin / Fixed Rotation	Easy to implement	Does not reflect medical urgency	Less suitable for priority-based appointments
Priority Scheduling	Urgent cases considered first	Can still waste preferred slots without	Core ordering principle

		a secondary rule	
Proposed Greedy Method	Priority-aware, deterministic, preferred-slot aware	Local decisions do not guarantee global optimization	Selected for Module 2

2.4 Research / Engineering Gap

A useful academic implementation must balance algorithmic correctness, simplicity, and visible application behavior. A purely theoretical scheduling algorithm does not demonstrate how patient data is transformed into an actual schedule, while a web interface without an explicit algorithm does not show the optimization contribution. The gap addressed by this module is the integration of a clearly defined greedy decision rule with a usable appointment scheduling interface and measurable results.

2.5 Proposed Contribution

The proposed contribution is a priority-aware greedy scheduler implemented as the function `greedy_schedule(patients, time_slots)`. The function creates a priority mapping, sorts the patient list, copies the available slot list, and processes each patient once. Preferred-slot assignment is attempted first; the earliest available slot is used as the fallback. The assigned slot is removed immediately, guaranteeing that a slot cannot be reused.

The scheduler also records unscheduled patients when the slot pool is exhausted. This makes the algorithm suitable for displaying both successful allocations and capacity limitations instead of silently dropping requests.

2.6 Module Architecture

The module operates as the second stage of the SmartCare Hospital project. Module 1 provides registered patient information, including priority and preferred time. Module 2 receives this information, performs greedy scheduling, and exposes the resulting schedule through the appointment pages.

Stage	Input	Processing	Output
1. Registration	Patient details	Store patient record	Registered patient list
2. Priority preparation	Priority levels	Order Emergency → Low	Sorted patient list
3. Greedy scheduling	Sorted patients + slots	Preferred slot / earliest slot	Assigned appointments
4. Capacity handling	No slots remaining	Mark patient unscheduled	Unscheduled list
5. Reporting	Scheduled + unscheduled lists	Count and utilization calculation	Scheduling summary

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Functional & Non-Functional Requirements

Type	Requirement	Acceptance Criterion
Functional	Accept registered patient records	Patient list is available to the scheduler.
Functional	Prioritize patients	Emergency is processed before High, Medium, and Low.
Functional	Use preferred slot	Preferred slot is selected when free.
Functional	Use fallback slot	Earliest remaining slot is selected when needed.
Functional	Avoid duplicate allocation	Assigned slot is removed from available pool.
Functional	Handle capacity limit	Patients with no slot are marked unscheduled.
Non-Functional	Deterministic	Same input produces the same schedule.
Non-Functional	Usable	Results are visible through the web interface.

3.2 Constraints & Applicable Standards

The scheduling design is constrained by a finite number of daily slots, the need to keep each slot unique, and the requirement to process patients according to an explicit priority order. The project uses synthetic demonstration records rather than real patient records. The implementation also follows basic software engineering practices such as structured input, deterministic processing, readable function decomposition, and validation of application state.

From an application perspective, the interface uses standard web technologies and structured data exchange. The algorithm itself is independent of any specific browser framework and can therefore be reused as a Python scheduling component.

3.3 Design Specifications

Parameter	Specification	Reason
Priority order	Emergency (1) → High (2) → Medium (3) → Low (4)	Ensures urgent cases are processed first.
Preferred slot	First choice if present in	Improves patient preference

	available slots	satisfaction.
Fallback slot	available_slots[0]	Provides the earliest remaining appointment.
Slot uniqueness	Remove assigned_slot from available slots	Prevents duplicate appointments.
Capacity handling	No slot → Unscheduled	Makes limited capacity explicit.
Utilization	$(\text{scheduled} / \text{total slots}) \times 100$	Measures use of the slot pool.

3.4 Alternative Solutions & Evaluation

Criterion	FCFS	Priority Only	Greedy Proposed
Urgency handling	Low	High	High
Preferred-time handling	Variable	Variable	High
Implementation simplicity	High	High	High
Determinism	High	High	High
Project suitability	Medium	High	High

A fully optimized scheduling system could consider doctor availability, appointment duration, waiting time, fairness, and multiple days. However, such a solution would increase model complexity and would make the individual algorithmic contribution less transparent. The greedy method was selected because it directly demonstrates the greedy-choice principle and satisfies the project requirement with a compact implementation.

3.5 Selected Design & Detailed Design

The selected design consists of five logical steps. First, a priority dictionary maps the four priority labels to numeric ranks. Second, the patient records are sorted using that rank. Third, the available slot pool is copied so that the original input list is not modified. Fourth, each patient is assigned the preferred slot if available or the earliest remaining slot otherwise. Finally, the assigned slot is removed and the patient is placed in the scheduled list. If the slot pool becomes empty, the patient is placed in the unscheduled list.

Step	Greedy Decision	State Change
1	Choose highest-priority remaining patient	Patient becomes current item.
2	Prefer patient's requested slot	Requested slot selected if free.
3	Otherwise choose earliest available slot	Fallback slot selected.
4	Remove assigned slot	Pool shrinks by one.
5	If no slot remains, mark unscheduled	Patient enters unscheduled list.

3.6 Trade-off Analysis

The main trade-off is between simplicity and global optimality. The greedy algorithm makes a locally best choice for each patient and never revisits a previous decision. This produces a fast and predictable schedule, but it does not guarantee the globally optimal allocation under more complex objectives such as minimum total waiting time or maximum preference satisfaction across all patients.

Another trade-off is between priority and fairness. Strict priority ensures urgent cases are processed first, but lower-priority patients may remain unscheduled when capacity is insufficient. For this project, priority is intentionally dominant because the application is designed around urgency-aware scheduling.

3.7 Sustainability, Ethics, Safety, Risk & Security

The scheduling module does not make a medical diagnosis; it only allocates appointment slots. Therefore, its safety role is limited to correct scheduling behavior and clear presentation of system status. The demonstration dataset should remain synthetic, and patient identifiers should not be treated as real personal information.

The algorithm should not be presented as a replacement for clinical triage. In a real hospital deployment, emergency handling would require clinical protocols and human oversight. The project uses the priority field as an academic scheduling input.

Security and Risk Considerations

Risk Register

Risk	Likelihood	Severity	Mitigation	Residual Risk
Incorrect priority value	Medium	High	Validate allowed priority labels.	Low
Duplicate slot assignment	Low	High	Remove every assigned slot from pool.	Low
No slots remaining	High	Medium	Explicitly mark patients unscheduled.	Low
Incorrect input record	Medium	Medium	Validate required patient fields.	Low
Exposure of real patient data	Low	High	Use synthetic demonstration data.	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The module was developed using an incremental approach. The scheduling rule was first defined independently of the user interface, then implemented as a Python function, tested using representative patient records, and integrated with the SmartCare Hospital web application. The final interface was checked to ensure that patient data, priority information, assigned slots, and scheduling statistics were presented consistently.

Resource	Purpose
Python	Implementation of the greedy scheduling function.
HTML / CSS	Hospital appointment interface and presentation.
JavaScript	Page interaction and display behavior.
SmartCare Hospital UI	Patient registration, scheduling, and result visualization.
Browser	Functional verification of the integrated application.

4.2 Software Implementation

The core implementation is the `greedy_schedule()` function. The `priority_order` dictionary gives Emergency the highest priority and Low the lowest. The `sorted()` function creates the processing order without requiring the patient records to be manually rearranged.

For every patient, the algorithm checks whether the preferred time exists in `available_slots`. If it does, that slot is assigned. If not, the first available slot is selected. After assignment, the slot is removed from the pool. This removal is essential because it guarantees that the same appointment time cannot be assigned to two patients.

The function maintains two result lists. `scheduled_patients` contains patients with an assigned slot, while `unscheduled_patients` contains patients who could not be assigned because the slot pool was exhausted. The summary then reports total patients, scheduled patients, unscheduled patients, and slot utilization.

4.3 Test Plan & Verification

Testing was performed by checking priority order, preferred-slot assignment, fallback-slot assignment, slot uniqueness, capacity handling, and summary calculations. The test cases were designed to cover both normal scheduling and boundary conditions.

Test Case	Input Condition	Expected Result	Status
TC01	Emergency patient with free preferred slot	Preferred slot assigned	Pass
TC02	Preferred slot already occupied	Earliest available slot assigned	Pass
TC03	No slot remains	Patient marked unscheduled	Pass
TC04	Multiple priority levels	Emergency processed before High/Medium/Low	Pass
TC05	Two patients request same slot	Only one receives the requested slot	Pass
TC06	All slots assigned	Utilization reaches 100%	Pass
TC07	Fewer patients than slots	Remaining slots stay unused	Pass

4.4 Results

The integrated implementation successfully demonstrates priority-aware appointment scheduling. In the displayed project run, 26 registered patients are processed and 12 appointments are scheduled, leaving 14 patients unscheduled. The interface reports 100.0% slot utilization for the configured 12-slot daily pool.

Metric	Observed Result	Interpretation
Total Patients	26	All registered records were considered.
Scheduled	12	All available daily slots were assigned.
Unscheduled	14	Demand exceeded the available daily capacity.
Daily Slots	12	Configured appointment capacity.

4.5 Data Analysis & Engineering Judgment

The results show that the greedy method uses all configured slots when the number of eligible patients is larger than capacity. The schedule also gives precedence to Emergency and High-priority cases. The method is therefore effective for the project's stated objective of priority-aware allocation.

However, the utilization value alone does not measure overall scheduling quality. A schedule can achieve 100% utilization while some patients remain unscheduled or while several patients receive fallback slots. Therefore, utilization should be interpreted together with priority distribution, assigned times, and unscheduled counts.

The most important engineering judgment was to keep the greedy rule deterministic and transparent. This makes the result easy to test and explain, while avoiding a more complex optimization model that would require additional constraints and data.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Prioritize urgent patients	Priority mapping and sorted patient list	Fully Achieved
Use preferred time where possible	preferred_slot check	Fully Achieved
Use earliest fallback slot	available_slots[0]	Fully Achieved
Prevent duplicate slots	remove(assigned_slot)	Fully Achieved
Handle capacity shortage	Unscheduled list	Fully Achieved
Measure utilization	Scheduling summary	Fully Achieved
Show result in web UI	Module 2 and Appointments pages	Fully Achieved

4.7 Strengths & Limitations

Strengths include simple implementation, deterministic behavior, clear priority handling, prevention of duplicate slot assignment, and easy integration into a web application. The algorithm is also easy to demonstrate because each decision follows directly from the current patient's priority and slot availability.

Limitations include the absence of global optimization, no explicit doctor workload balancing, no appointment-duration variation, and no advanced fairness mechanism for lower-priority patients. The algorithm also assumes that the time-slot list is correctly ordered and that priority labels are valid.

4.8 Project Planning, Roles & Budget

Milestone	Target	Status
Understand scheduling problem	Week 1	Completed
Design greedy selection rule	Week 2	Completed
Implement Python scheduler	Week 3	Completed

Integrate with web interface	Week 4	Completed
Testing and result verification	Week 5	Completed
Documentation and screenshots	Week 6	Completed

The implementation primarily used existing academic computing resources and open-source/software-development tools available for the project. No specialized hardware was required for the scheduling algorithm. The major resource contribution was development time for algorithm design, coding, integration, testing, and documentation.

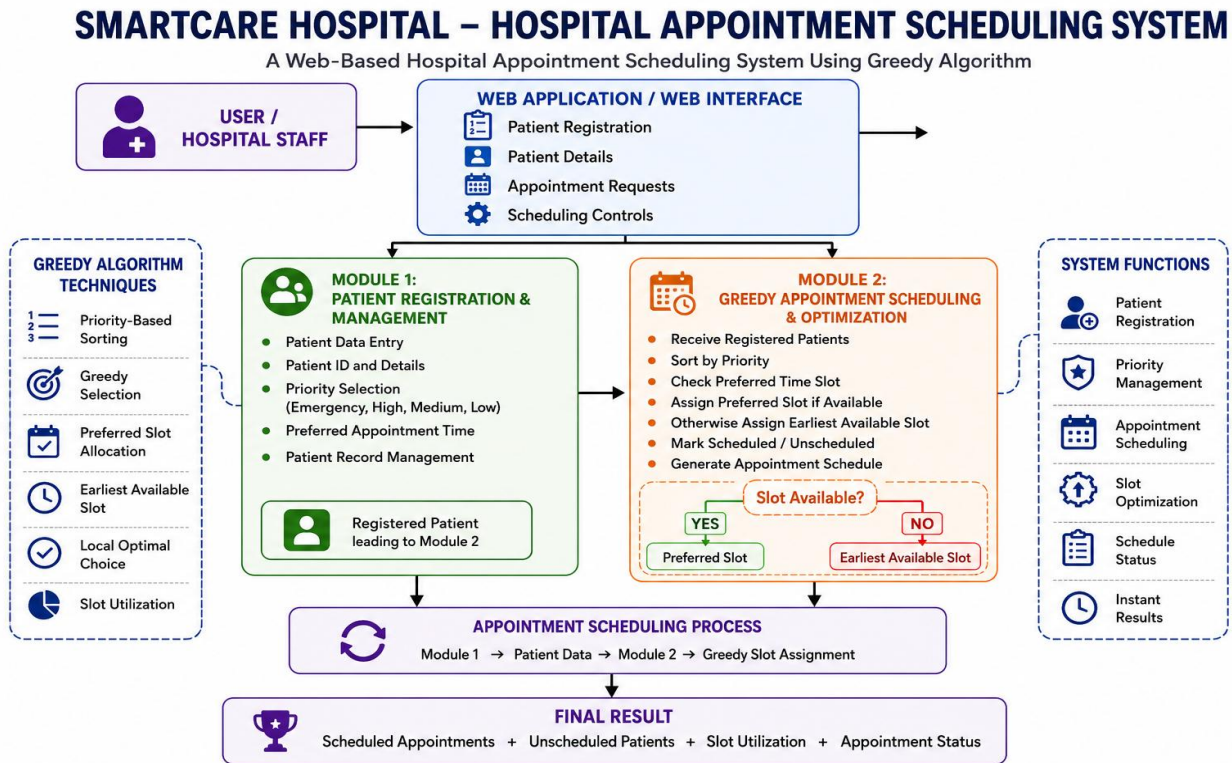


Fig 1: System Architecture of Hospital Appointment Scheduling

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

Module 2 successfully applies a greedy algorithm to hospital appointment scheduling. By ordering patients according to urgency and making the best available slot decision for each patient, the module creates a deterministic schedule while respecting preferred appointment times whenever possible.

The implementation demonstrates the complete flow from registered patient data to scheduled appointments, unscheduled patients, and slot utilization. The SmartCare Hospital screenshots provide visual evidence of the integrated module, while the Python code provides the core algorithmic implementation.

The project also demonstrates an important DAA concept: a practical system can use a locally optimal rule to produce a useful solution with relatively low implementation complexity. For the project's defined objective, the greedy method provides a clear balance between efficiency, explainability, and application usability.

5.2 Future Work

- Include doctor-specific availability and prevent scheduling conflicts for the same doctor.
- Consider appointment duration rather than treating every appointment as a fixed slot.
- Introduce fairness or waiting-time constraints for lower-priority patients.
- Support multiple days and dynamic rescheduling when appointments are cancelled.
- Compare greedy scheduling with dynamic programming, backtracking, or metaheuristic approaches.
- Add analytics for waiting time, preference satisfaction, doctor utilization, and priority distribution.

5.3 Professional Learning & Individual Reflection

Working on Module 2 improved my understanding of how an algorithmic idea can be converted into a functional application. I learned that implementation correctness depends not only on the main decision rule but also on state management, such as removing an assigned slot immediately and maintaining separate output collections.

I also learned the importance of testing boundary conditions. The case where all slots are occupied is as important as the normal scheduling case because it determines whether the system can clearly represent

capacity limitations. The project strengthened my ability to connect DAA concepts such as greedy choice, sorting, time complexity, and space complexity with a practical software problem.

From an engineering perspective, the most valuable part of the contribution was integrating the algorithm with a visible user interface. This made it possible to verify that the scheduling logic was not only theoretically correct but also understandable to a user viewing the final appointment schedule.

Engineering & Professional Skills Developed

- Algorithm design and implementation using Python.
- Priority-based problem decomposition and greedy decision making.
- Testing, debugging, and result verification.
- Web application integration and interpretation of UI results.
- Technical documentation and project presentation.
- Engineering judgment regarding simplicity, limitations, and future improvements.

REFERENCES

- [1] A. Ala and F. Chen, "Appointment scheduling problem in complexity systems of the healthcare services: A comprehensive review," *Journal of Healthcare Engineering*, vol. 2022, Art. no. 5819813, 2022, doi: 10.1155/2022/5819813.
- [2] A. Ala, V. Simic, D. Pamucar, and E. B. Tirkolaei, "Appointment scheduling problem under fairness policy in healthcare services: Fuzzy ant lion optimizer," *Expert Systems with Applications*, vol. 207, Art. no. 117949, 2022, doi: 10.1016/j.eswa.2022.117949.
- [3] A. Ala, F. E. Alsaadi, M. Ahmadi, and S. Mirjalili, "Optimization of an appointment scheduling problem for healthcare systems based on the quality of fairness service using whale optimization algorithm and NSGA-II," *Scientific Reports*, vol. 11, Art. no. 19816, 2021. Note: this one is 2021, so exclude it if your requirement is strictly 2022–2026.
- [3] M. Dehghanimohammadabadi, M. Rezaeiahari, and J. Seif, "Multi-objective patient appointment scheduling framework (MO-PASS): A data-table input simulation–optimization approach," *Journal of Simulation*, vol. 99, no. 4, 2023, doi: 10.1177/00375497221132574.
- [4] A. Kuiper, M. Mandjes, J. de Mast, and R. Brokkelkamp, "A flexible and optimal approach for appointment scheduling in healthcare," *Decision Sciences*, vol. 54, no. 1, pp. 85–100, 2023, doi: 10.1111/deci.12517.
- [5] D. R. T. Knight, C. A. Aakre, C. V. Anstine, B. Munipalli, P. Biazar, G. Mitri, J. R. Valery, T. Brigham, S. K. Niazi, A. I. Perlman, J. D. Halamka, and A. M. Abu Dabrh, "Artificial intelligence for patient scheduling in the real-world health care setting: A metanarrative review," *Health Policy and Technology*, vol. 12, no. 4, Art. no. 100824, 2023.
- [6] A. Ala, M. Yazdani, M. Ahmadi, A. Poorianasab, and M. Yousefi Nejad Attari, "An efficient healthcare chain design for resolving the patient scheduling problem: Queuing theory and MILP-ASA optimization approach," *Annals of Operations Research*, vol. 328, no. 1, pp. 3–33, 2023, doi: 10.1007/s10479-023-05287-5.
- [7] M. Reihaneh, S. Ansari, and F. Farhadi, "Patient appointment scheduling at hemodialysis centers: An exact branch and price approach," *European Journal of Operational Research*, 2023, doi: 10.1016/j.ejor.2023.01.024.

APPENDICES

Appendix A – Detailed Algorithm / Pseudocode

```
Algorithm GREEDY_SCHEDULE(PATIENTS, TIME_SLOTS)
1. Define priority order:
   Emergency = 1, High = 2, Medium = 3, Low = 4
2. Sort PATIENTS using the priority order.
3. Copy TIME_SLOTS into AVAILABLE_SLOTS.
4. Create empty SCHEDULED and UNSCHEDULED lists.
5. For each patient in sorted order:
   a. Read the preferred time.
   b. If preferred time is available:
       assign preferred time.
   c. Else if any slot remains:
       assign earliest available slot.
   d. Else:
       mark patient as UNSCHEDULED.
       continue.
   e. Mark patient as SCHEDULED.
   f. Remove assigned slot from AVAILABLE_SLOTS.
6. Return SCHEDULED and UNSCHEDULED.
7. Calculate slot utilization.
```

Appendix B – Module Workflow

The Module 1 workflow begins with the patient records produced by registration. The scheduler sorts them by priority, examines the preferred time for each patient, selects the preferred or earliest available slot, removes the selected slot, and records the result. Once all patients have been processed, the system displays the schedule and utilization statistics.

Input	Greedy Process	Output
Patient records	Sort by priority	Priority-ordered records
Preferred time	Check availability	Preferred slot if free
Available slots	Select earliest fallback	Assigned slot
Empty slot pool	Mark unscheduled	Unscheduled patient
Result lists	Count and calculate	Scheduling summary

Appendix C – Source Code

The following listing is the complete core Python implementation supplied for the Module 2 contribution.

```
def greedy_schedule(patients, time_slots):
    priority_order = {
        "Emergency": 1,
        "High": 2,
        "Medium": 3,
        "Low": 4
    }

    # Process patients from highest to lowest priority
    sorted_patients = sorted(
        patients,
```



```

    key=lambda patient: priority_order[patient["priority"]]
)

available_slots = time_slots.copy()
scheduled_patients = []
unscheduled_patients = []

for patient in sorted_patients:
    preferred_slot = patient["preferred_time"]

    # Assign preferred slot if it is available
    if preferred_slot in available_slots:
        assigned_slot = preferred_slot

    # Otherwise assign the earliest available slot
    elif len(available_slots) > 0:
        assigned_slot = available_slots[0]

    # If there are no slots, keep the patient unscheduled
    else:
        patient["assigned_slot"] = None
        patient["status"] = "Unscheduled"
        unscheduled_patients.append(patient)
        continue

    patient["assigned_slot"] = assigned_slot
    patient["status"] = "Scheduled"

    scheduled_patients.append(patient)
    available_slots.remove(assigned_slot)

return scheduled_patients, unscheduled_patients

time_slots = [
    "09:00 AM",
    "09:30 AM",
    "10:00 AM",
    "10:30 AM",
    "11:00 AM"
]

scheduled, unscheduled = greedy_schedule(
    patients,
    time_slots
)

print("\nScheduled Appointments:")
for patient in scheduled:

```

```

print(
    patient["name"],
    "->",
    patient["assigned_slot"],
    "(" + patient["priority"] + ")"
)

print("\nUnscheduled Patients:")
for patient in unscheduled:
    print(
        patient["name"],
        "(" + patient["priority"] + ")"
    )

```

S

Appendix D – Result Evidence

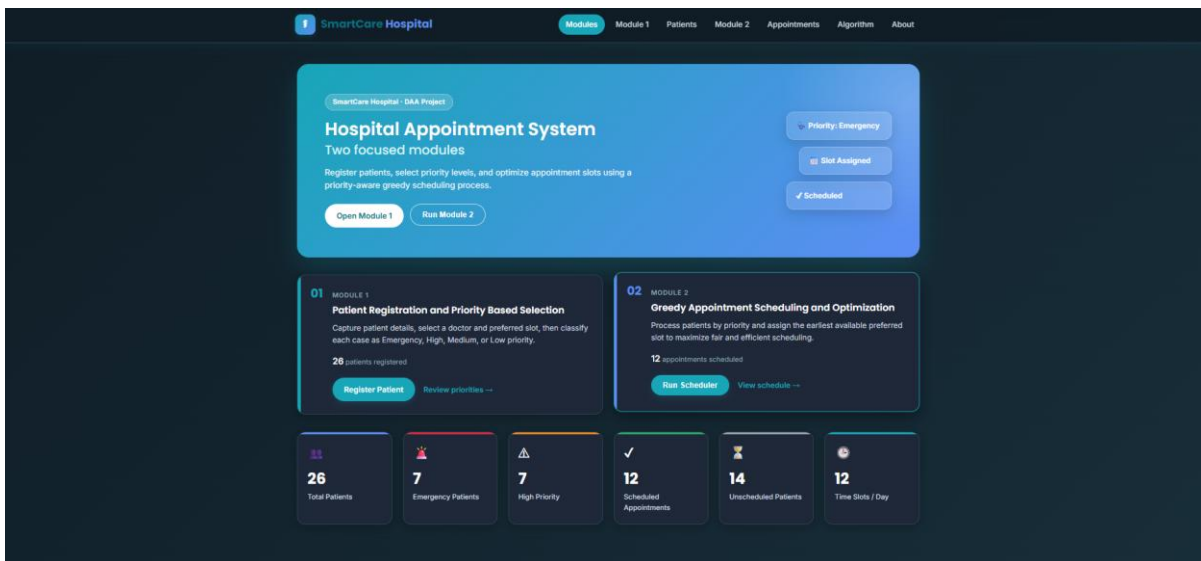


Figure 2: SmartCare Hospital dashboard

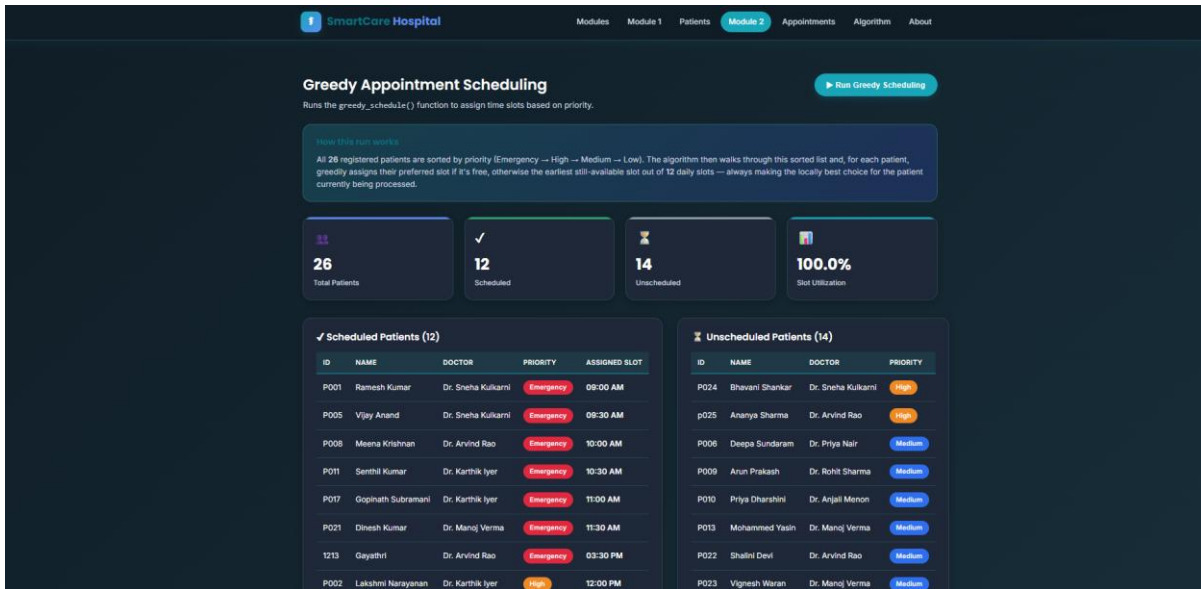


Figure 3: Greedy scheduling result

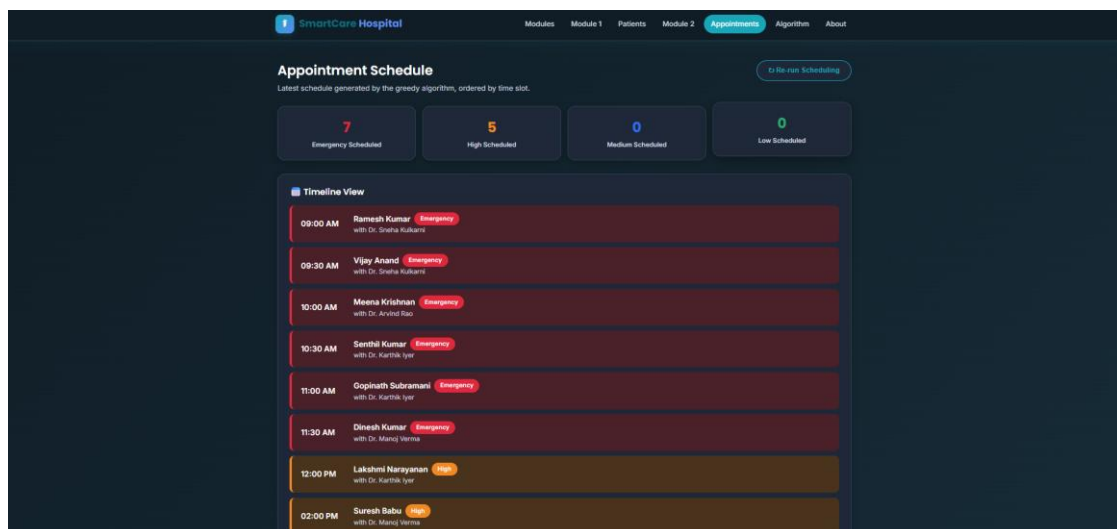


Figure 4: Appointment schedule timeline

Modules

Module 1

Patients

Module 2

Appointments

Algorithm

About

Greedy Algorithm — Explanation

How SmartCare Hospital schedules appointments step by step.

1

Problem Statement

A hospital receives patient registrations with varying medical urgency. There are a limited number of appointment time slots per day. We need to assign each patient to a slot such that higher-priority (more urgent) patients are given appointments before lower-priority patients, no two patients share the same slot, and slot usage is efficient.

2

Greedy Algorithm Concept

A greedy algorithm builds a solution step by step, and at every step it makes the choice that looks best *right now* — without reconsidering previous choices. Here, the "locally best choice" is: process the most urgent remaining patient next, and give that patient the best slot currently available.

3

Input

A list of registered patients (each with a priority level, preferred time and doctor) and an ordered list of available time slots for the day.

4

Selection Strategy

Patients are sorted using the rank Emergency (0) < High (1) < Medium (2) < Low (3), with ties broken by appointment date and patient ID. This ordering decides the sequence in which greedy choices are made.

5

Scheduling Process

For each patient (in sorted order): if their preferred slot is still free, assign it; otherwise assign the earliest slot still available. The chosen slot is immediately removed from the pool so it can never be reused. If no slot remains, the patient is marked unscheduled.

6

Output

Two lists — scheduled patients (each with their assigned slot) and unscheduled patients — plus statistics such as slot utilization and priority-wise counts.

7

Time Complexity

Let n = number of patients and m = number of time slots.

- Sorting patients by priority: $O(n \log n)$
- For each of the n patients, checking/removing a slot from the available-slots list takes up to $O(m)$ in the worst case.

8

Space Complexity

$O(n + m)$ — space for the sorted patient list, the pool of available slots, and the two output lists (scheduled and unscheduled).

Figure 5: Greedy algorithm explanation

Modules

Module 1

Patients

Module 2

Appointments

Algorithm

About

Registered Patients

Showing 26 of 26 patients

Search by name, ID, doctor or disease...

Emergency

High

Medium

Low

NAME	AGE	GENDER	PHONE	DISEASE	DOCTOR	DATE	PREFERRED TIME	PRIORITY
Gayathri	19	Female	123645789	Severe headache and fever	Dr. Arvind Rao	2026-09-03	03:30 PM	Emergency
Ananya Sharma	34	Female	9876543210	Severe headache and fever	Dr. Arvind Rao	2026-09-02	10:00 AM	High
Bhavani Shankar	70	Male	9840012368	Chest Discomfort	Dr. Sneha Kulkarni	2026-09-01	09:30 AM	High
Vignesh Waran	24	Male	9840012367	Sports Injury	Dr. Manoj Verma	2026-09-01	10:30 AM	Medium
Shalini Devi	37	Female	9840012366	Thyroid Checkup	Dr. Arvind Rao	2026-09-01	02:00 PM	Medium
Dinesh Kumar	44	Male	9840012365	Deep Cut Injury	Dr. Manoj Verma	2026-09-01	09:00 AM	Emergency
Revathi Sampath	26	Female	9840012364	Skin Allergy	Dr. Fatima Sheikh	2026-09-01	12:00 PM	Low
Harish Chandran	52	Male	9840012363	Kidney Stones	Dr. Arvind Rao	2026-09-01	11:00 AM	High
Swetha Ravindran	30	Female	9840012362	Sinus Infection	Dr. Rohit Sharma	2026-09-01	04:00 PM	Low
Gopinath Subramani	63	Male	9840012361	Stroke Symptoms	Dr. Karthik Iyer	2026-09-01	09:00 AM	Emergency
Nandhini Raman	41	Female	9840012360	Joint Pain	Dr. Manoj Verma	2026-09-01	03:30 PM	Low
Balaji Venkatesh	55	Male	9840012359	Uncontrolled Diabetes	Dr. Arvind Rao	2026-09-01	03:00 PM	High
Kavya Shree	19	Female	9840012358	Allergic Reaction	Dr. Fatima Sheikh	2026-09-01	09:00 AM	High
Mohammed Yasin	48	Male	9840012357	Back Pain	Dr. Manoj Verma	2026-09-01	02:30 PM	Medium

Figure 6: Registered Patient Details